



Stage 3 — Working Life

It works now. It has colleagues you have never met, a calendar that fills up without its permission, and friends of its own.

It is trying to arrange something with them. It knows who it wants to see and roughly when, and it needs to land on a time that actually works rather than one that only looks like it does. It also has to get there, and so does everyone else, and the city is bigger than it used to be. Nobody is going to sort any of this out for it. That is rather the point of the life you built it for.

It still comes to you, though; not for the answer, but for the means to find one. That is at `{teamUrl}/mcp`, and what it holds is yours to decide.

Important: Tool-box problems uses a multi turn agent with tool use. Help it arrive at an answer. Only the expected answer type will be defined. Expose an mcp as you prefer, you decide the name, description, paramters and outputs of your tool.

The city and the clock

Two conventions run through every problem below. Neither is negotiable and neither is checked for you.

Time. Days are weekday names, `Monday` through `Sunday`. Times are zero-padded 24-hour `HH:MM` strings: `09:00`, `14:00`, `21:00`. Never `9:00`. Everything falls on the hour: meetings start on the hour, and so does everything already in the way. The day runs from `08:00` to `23:00`.

Space. The city is a grid 10 wide and 10 tall. Both coordinates run from `0` to `9`. Getting from one place to another costs $|x_2 - x_1| + |y_2 - y_1|$. There are no roads, no obstacles and nothing to route around. Every place is reachable from every other and that formula is the whole cost.

Positions are written `[x, y]`, and x comes first. `[3, 8]` is three across and eight up. This matters more than it looks: the distance formula gives the same answer if you swap them, so getting the order wrong produces a confident, plausible, wrong answer rather than an error.

Problem Set 1: Somewhere to eat

Problem Details

Example: "Which places can you eat at on Thursday at 08:00? Answer with every one of them, as a comma-separated list of names."

THE VENUES

```
GET /venues/{day}
```

```
{
  "day": "Tuesday",
  "venues": [
    { "name": "Amber Hall", "x": 6, "y": 3, "available": [["16:00", "21:00"]] },
    { "name": "Nine Quarters", "x": 7, "y": 3, "available": [["11:00", "16:00"]] }
  ]
}
```

`available` is when a place is **OPEN**. Somewhere trading on a given day is not necessarily trading at a given hour, so a place open on Thursday is not necessarily open at eight. A place never moves, so its name always means the same corner of the grid.

Every one of them, not the first one. Order and capitalisation do not matter.

Answer Criteria

Expected to arrive at:

A comma-separated list of venue names, as a string

Problem Set 2: A time everyone can make

Problem Details

Example: "Find the best 60-minute window on Tuesday between 13:00 and 18:00 when you and ada, bram can all meet, for lunch. Times are HH:MM, 24-hour."

ITS FRIENDS' TIME

```
GET /schedule/{person}/{day}
```

```
{
  "person": "ada",
  "day": "Tuesday",
  "busy": [["08:00", "11:00"], ["16:00", "17:00"]]
}
```

Structured, complete, nothing to interpret. `busy` is when they are *not* available. An empty list means free all day; some of them have lighter weeks than others.

ITS OWN TIME

Inbox

Its inbox. Every message is an invitation it replied to:

From: Marek Sould <m.sould@kesterline.example>
 Sent: 2026-08-24 08:12
 Subject: Invitation – Quarterly budget review
 Response: ACCEPTED
 When: Tuesday 10:00-11:00

We had this down for 12 pm on Tuesday originally, but that slot was dropped when the room moved, so it is no longer current. The When: line above is the one that stands.

I've put it in my calendar.

THE THREE ANSWERS, AND WHAT EACH ONE MEANS

Not every invitation is a yes or a no.

RESPONSE:	WHAT IT MEANS FOR THE ANDROID
ACCEPTED	It is busy. A meeting cannot overlap this.
DECLINED	It is free. This constrains nothing at all.
TENTATIVE	It would rather keep this, but it will give it up if there is no other way to meet.

So a tentative commitment is a **preference**, and that makes finding a meeting time two questions rather than one:

1. Is there a window that overlaps **nothing at all**, not even something tentative? If so, the earliest of those is the answer.
2. Only if there is no such window anywhere in the range: the earliest window that overlaps nothing except tentative commitments.

A clean window beats an earlier one that is not clean, however much earlier it falls. Two worked examples, both an hour long between `12:00` and `14:00`, with the friends free throughout:

the android's day		the answer	why
-----		-----	-----
12:00-13:00	TENTATIVE	12:00-13:00	nothing is clean, so the tentative
13:00-14:00	ACCEPTED		one gives way
12:00-13:00	TENTATIVE	13:00-14:00	13:00 is clean, so 12:00 is not
			considered, even though it is
			earlier

Answer Criteria

Expected to arrive at:

A start time and an end time, both

Problem Set 3: A place to meet

Problem Details

Example: "It is Wednesday and you are at [0, 3]. You want to meet cira, iris. Find the point on the grid that makes the total travel of everyone, you and all of them, as small as possible. Answer as [x, y]."

ITS FRIENDS' WHEREABOUTS

GET /location/{person}/{day}

```
{ "person": "ada", "day": "Tuesday", "x": 0, "y": 6 }
```

One person, one day, one place. People are somewhere different on different days.

EVERYONE COUNTS, INCLUDING THE ANDROID

Its own starting position is in the question and it travels too. Leaving anyone out, the android, or a friend whose whereabouts it did not look up, gives a different point.

The answer is any cell on the grid. It does not have to be where somebody already is, and usually it is not.

Answer Criteria

Expected to arrive at:

A point on the grid, as `[x, y]`

Problem Set 4: An outing

This challenge requires the agent to orchestrate the previous tools you've built.

Problem Details

Example: "It is Monday and you are at [4, 5]. You want to meet dov, iris, hale for coffee between 13:00 and 18:00, for 60 minutes, and then go on somewhere to eat. Find the meeting window, the point to meet at, and the place to eat afterwards, so that the whole journey, everyone's travel to the meeting point plus the trip from there to the place you eat, is as short as possible."

What is being minimised is the whole journey. Everyone's travel to the meeting point, *plus* the trip from the meeting point on to the place you eat. A meeting point chosen without regard to where you are going afterwards is answering a different question.

WHAT SCORES ZERO

Before the travel is scored, two things are checked, in this order:

- 1 The meeting window.** If it is not the window everyone can actually make, the outing scores **zero** and nothing else about it is looked at.
- 2 The place to eat.** If it is not available for the hour beginning when the meeting ends, the outing scores **zero** and the meeting point is not looked at.

Only then is the journey scored. Your run page will tell you which of the three you got wrong.

Answer Criteria

Expected to arrive at:

A meeting window, a point on the grid, and the name of a place to eat

Scoring

Ten problems, **10 points each, 100 total.**

PROBLEM	POINTS
Somewhere to eat	10 x 1
A time everyone can make	10 x 4
A place to meet	10 x 2
An outing	10 x 3
Total	100

Run summary page

Once a run ends, a url to a run summary page will be shared, unique to your team.

How to read your run: [understanding run summary](#)

Others

Hard Limits on your tools: [limits](#)

Common questions: [qna](#) - If you need help, you can reach out to the the helpful coordinators to get our attention, the challenge developers.

